

PRACTICA 6 DE PROGRAMACIÓN DE SISTEMAS CONCURRENTES Y DISTRIBUIDOS

Índice:

Página 1 y 2. Como he resuelto el código

Página 3: Problemas y soluciones

Página 4: Ejemplo ejecuciones

Como se ha resuelto el ejercicio:

Para resolver el servidor del ahorcado, he hecho las siguientes cosas:

El código implementa un servidor para el juego del ahorcado utilizando sockets en Java. Al inicio, se definen constantes clave como el puerto donde el servidor estará escuchando conexiones (PUERTO = 45000) y el número máximo de intentos permitidos que son en este caso 10. También se declaran las variables `socketServer` y `socket`, que representan el servidor y una conexión específica con un cliente.

En la configuración inicial del servidor, se crea un `ServerSocket` que escucha el puerto especificado al principio, hasta que un cliente se conecte. Una vez que el cliente establece la conexión con el servidor, se genera un objeto `Socket`, lo que permite la comunicación entre el cliente y el servidor. Se configuran flujos de entrada y salida utilizando `BufferedReader` y `BufferedWriter` para facilitar la lectura y escritura de datos entre el servidor y el cliente.

Luego el juego se inicializa definiendo una palabra fija que será la que cliente deberá adivinar en este caso la palabra es: HOLA, la variable `palabraAdivinadaMomento`, va a almacenar una versión oculta de la palabra con guiones (-) para las letras aún no adivinadas. Todo se consigue con el método `inicializarPalabraAdivinadaMomento`, método que se encarga de reemplazar todas las letras de la palabra con guiones. Además, de establecer un contador para los intentos fallidos.

El núcleo del juego reside en un bucle `while (true)`. En cada iteración, el servidor envía al cliente el estado actual de la palabra parcialmente adivinada y los intentos restantes. A continuación, recibe una letra del cliente y el servidor verifica si la letra indicada está en la palabra. Si la letra es incorrecta, el contador se reduce en 1, mientras que, si es correcta, se actualiza `palabraAdivinadaMomento` mediante el método `actualizarPalabraAdivinadaMomento`, que construye una nueva versión de la palabra mostrando las letras correctamente adivinadas y dejando guiones en las posiciones restantes.

El juego termina bajo dos condiciones: si el cliente adivina completamente la palabra, el servidor envía un mensaje de victoria; si el cliente supera el número máximo de intentos fallidos, el servidor informa de la derrota y revela la palabra completa. En ambos casos, el servidor envía un mensaje final con la palabra clave FIN para indicar el cierre de la conexión.

Finalmente, el servidor realiza la limpieza de los recursos utilizados cerrando tanto el `socket` del cliente como el `ServerSocket`. Esto asegura que no queden conexiones abiertas y libera los recursos del sistema. Este diseño combina de manera eficiente la lógica del juego y la comunicación entre cliente y servidor.

Para resolver el cliente del ahorcado, he hecho las siguientes cosas:

En primer lugar, se establecen las configuraciones iniciales. El cliente define la dirección IP y el puerto para conectarse al servidor. Estos valores deben coincidir con los configurados en el servidor. El cliente crea un objeto Socket para establecer la conexión con el servidor. Si la conexión es exitosa, el programa muestra un mensaje confirmando la conexión y prepara los flujos de entrada y salida para la comunicación.

El cliente utiliza flujos para enviar y recibir información. Con `BufferedReader` y `BufferedWriter`, se configuran los flujos de entrada y salida del socket, lo que facilita la lectura y escritura de datos. Además, se declara una constante para identificar el fin del juego, se inicializan variables como `palabraAdivinadaMomento` para almacenar la información recibida del servidor y `Scanner` teclado para gestionar la entrada del jugador.

En el bucle principal, el cliente espera recibir del servidor la palabra parcialmente adivinada y el número de intentos restantes. Esta información se lee con `br.readLine()` y se almacena en `palabraAdivinadaMomento`. Si el mensaje recibido es la constante `FIN`, significa que el juego ha terminado, y el cliente muestra un mensaje de despedida antes de salir del bucle.

El cliente muestra al jugador el estado actual de la palabra y solicita una letra nueva. Para garantizar una entrada válida, se utiliza un bucle `do-while` que se asegura de que el jugador introduzca exactamente una letra. Una vez validada, la letra se convierte a mayúsculas y se envía al servidor mediante `BufferedWriter`.

Finalmente, el cliente gestiona posibles excepciones que puedan ocurrir durante la conexión o la comunicación, como problemas de red o errores de entrada/salida, utilizando un bloque `try-catch`. Además, se asegura de cerrar automáticamente el socket al terminar el programa gracias al uso de un bloque `try-with-resources`, lo que libera los recursos utilizados. Este diseño permite al cliente interactuar de manera fluida con el servidor, respetando la lógica del juego.

Problemas y soluciones en el servidor:

Un problema importante en el servidor es la posible falta de sincronización en la comunicación con el cliente. Si el cliente no envía datos en el momento esperado, el servidor podría quedarse bloqueado esperando una entrada. Para solucionarlo, he implementado el uso de `BufferedReader` y `BufferedWriter`, que aseguran una comunicación más fiable al gestionar el envío y recepción de datos correctamente. Además, el método `flush` garantiza que los datos se envíen de inmediato.

Otro problema observado es la falta de manejo de letras inválidas. Si el cliente envía entradas no válidas, como cadenas vacías o caracteres especiales, esto podría romper la lógica del servidor. Aunque esta validación debería realizarse preferentemente en el cliente, el servidor se beneficia de su diseño robusto, ya que asume que las entradas serán siempre válidas.

El servidor también tenía una deficiencia en la manera de informar al cliente sobre el fin del juego. No existía un mecanismo claro para cerrar la conexión de forma ordenada. Esto lo he resuelto introduciendo un mensaje especial, `FIN`, que se envía al cliente al concluir el juego. Así, el cliente sabe cuándo terminar su ejecución.

Por último, la gestión de recursos fue un problema crítico. Si ocurría una excepción, los recursos del servidor (como los sockets) podían quedar abiertos, lo que lleva a fugas de recursos. Para abordar esto, se utilizó un bloque `finally` que garantiza el cierre adecuado de los recursos incluso en caso de error.

Problemas y soluciones en el cliente:

En el lado del cliente, un problema recurrente es la posibilidad de desconexión inesperada del servidor. Esto podría dejar al cliente bloqueado indefinidamente o generar excepciones no manejadas. Para resolver esto, he utilizado un bloque `try-catch` que captura cualquier error de entrada o salida, mostrando un mensaje informativo al usuario y cerrando la aplicación de forma ordenada.

Otro desafío importante es la validación de la entrada del usuario. Si el cliente permite entradas inválidas, como cadenas vacías o más de una letra, esto podría causar problemas en la lógica del servidor. Para evitarlo, he implementado un bucle `do-while` que valida que el usuario introduzca exactamente una letra antes de enviarla al servidor.

Un aspecto adicional es la falta de confirmación explícita de que los mensajes enviados han sido recibidos por el servidor. Aunque no se implementa una confirmación formal, se confía en el manejo confiable de datos mediante `BufferedWriter` y `BufferedReader`, que asegura que los datos se transmiten de forma adecuada.

Finalmente, el cliente también enfrentaba el problema de recursos abiertos al finalizar el juego. Si el socket no se cerraba adecuadamente, esto podía causar fugas de recursos. La solución fue emplear un bloque `try-with-resources`, que garantiza el cierre automático del socket al finalizar el programa, sin importar si ocurre un error.

Ejemplo de ejecuciones:

```
ServidorAhorcado (run) × ClienteAhorcado (run) ×  
  
run:  
Servidor esperando al cliente  
Cliente conectado  
BUILD SUCCESSFUL (total time: 18 seconds)  
|
```

```
ServidorAhorcado (run) × ClienteAhorcado (run) ×  
  
run:  
Conectado al servidor. Esperando respuesta...  
Esperando la respuesta del servidor...  
SE HA ADIVINADO HASTA EL MOMENTO: ---- | Intentos restantes: 10  
INDIQUE UNA NUEVA LETRA:  
QW  
INDIQUE UNA NUEVA LETRA:  
A  
Esperando la respuesta del servidor...  
SE HA ADIVINADO HASTA EL MOMENTO: ---A | Intentos restantes: 10  
INDIQUE UNA NUEVA LETRA:  
H  
Esperando la respuesta del servidor...  
SE HA ADIVINADO HASTA EL MOMENTO: H--A | Intentos restantes: 10  
INDIQUE UNA NUEVA LETRA:  
P  
Esperando la respuesta del servidor...  
SE HA ADIVINADO HASTA EL MOMENTO: H--A | Intentos restantes: 9  
INDIQUE UNA NUEVA LETRA:  
O  
Esperando la respuesta del servidor...  
SE HA ADIVINADO HASTA EL MOMENTO: HO-A | Intentos restantes: 9  
INDIQUE UNA NUEVA LETRA:  
L  
Esperando la respuesta del servidor...  
SE HA ADIVINADO HASTA EL MOMENTO: ♦Has ganado! La palabra era: HOLA  
INDIQUE UNA NUEVA LETRA:
```